Shopping List App – A Node.js and MongoDB Based Item Management System

ABSTRACT

The **Shopping List App** is a web-based backend application developed using **Node.js** and **MongoDB** that allows users to create and manage their shopping items efficiently. The platform enables users to add, view, update, and delete items in a structured and organized manner.

This application addresses the problem of managing daily shopping needs by providing a simple digital solution. The backend is built using **Express.js** for handling HTTP requests and **MongoDB** for storing item data.

The system demonstrates fundamental full-stack development concepts such as CRUD operations, RESTful API design, and database integration using modern web technologies.

PROJECT EXPLANATION

Introduction

Managing shopping items manually can be confusing and inefficient, especially when dealing with multiple items. People often forget items or lose track of their list.

The Shopping List App provides a simple digital platform where users can maintain a list of items they need to buy. It helps users stay organized and ensures nothing is missed while shopping.

Objective of the Project

- To develop a simple shopping list management system
- To implement backend development using Node.js
- To store and manage item data using MongoDB
- To perform CRUD operations (Create, Read, Update, Delete)
- To create a user-friendly interface for managing items

Technology Used

- Node.js – Backend runtime environment
- Express.js – Framework for building APIs
- MongoDB – NoSQL database for storing items
- Mongoose – ODM for MongoDB
- HTML, CSS, JavaScript – Frontend development

Working Principle

1.Users enter movie details:

- Item name
- Quantity

2. Data is sent to the backend through API requests.

3. The backend stores the data in MongoDB.

4. The system provides APIs to:

- Add new item
- View all item
- Update item details
- Delete item

5. The frontend fetches data and calculates the total expense dynamically.

Key Features

- Add items to shopping list
- View all items
- Edit item details
- Delete items
- Simple and responsive interface
- Real-time updates

Advantages

- Helps organize shopping efficiently
- Reduces chances of forgetting items
- Easy to use and beginner-friendly
- Fast data storage and retrieval
- Easily extendable with advanced features

Conclusion

The **Shopping List App** demonstrates how backend technologies like **Node.js** and **MongoDB** can be used to build a practical real-world application.

CODE IMPLEMENTATION :

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/shoppingDB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

// Routes
const itemRoutes = require("./routes/itemRoutes");
app.use("/items", itemRoutes);

// Start Server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Product.js

```
const mongoose = require("mongoose");

const itemSchema = new mongoose.Schema({
  name: String,
  quantity: Number
});

module.exports = mongoose.model("Item", itemSchema);
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Shopping List</title>

  <style>
    body {
      font-family: Arial;
      background: linear-gradient(135deg, #4b0082, #8a2be2);
      color: white;
      text-align: center;
      padding: 20px;
    }

    h1 {
      font-size: 40px;
    }

    input {
      padding: 10px;
```

```
    margin: 5px;
    border-radius: 8px;
    border: none;
    width: 200px;
  }

  button {
    padding: 10px 15px;
    margin: 5px;
    border: none;
    border-radius: 8px;
    background: gold;
    color: black;
    font-weight: bold;
    cursor: pointer;
  }

  .item {
    background: white;
    color: black;
    margin: 10px auto;
    padding: 10px;
    width: 300px;
    border-radius: 10px;
  }
 </style>
</head>

<body>

<h1>🛒 Shopping List</h1>

<input id="name" placeholder="Item name">
<input id="quantity" type="number" placeholder="Qty">
<br>
<button onclick="addItem()">Add Item</button>

<h2>Your Items</h2>
<div id="items"></div>

<script>
const API = "http://localhost:5000/items";

async function getItems() {
 const res = await fetch(API);
 const data = await res.json();

 const itemsDiv = document.getElementById("items");
 itemsDiv.innerHTML = "";

 data.forEach(item => {
  itemsDiv.innerHTML += `
   <div class="item">
    <h3>${item.name}</h3>
    <p>Quantity: ${item.quantity}</p>
    <button onclick="deleteItem('${item._id}')">Delete</button>
    <button onclick="editItem('${item._id}', '${item.name}', '${item.quantity}')">Edit</button>
   </div>
  `;
 });
}

async function addItem() {
 const name = document.getElementById("name").value;
 const quantity =
```

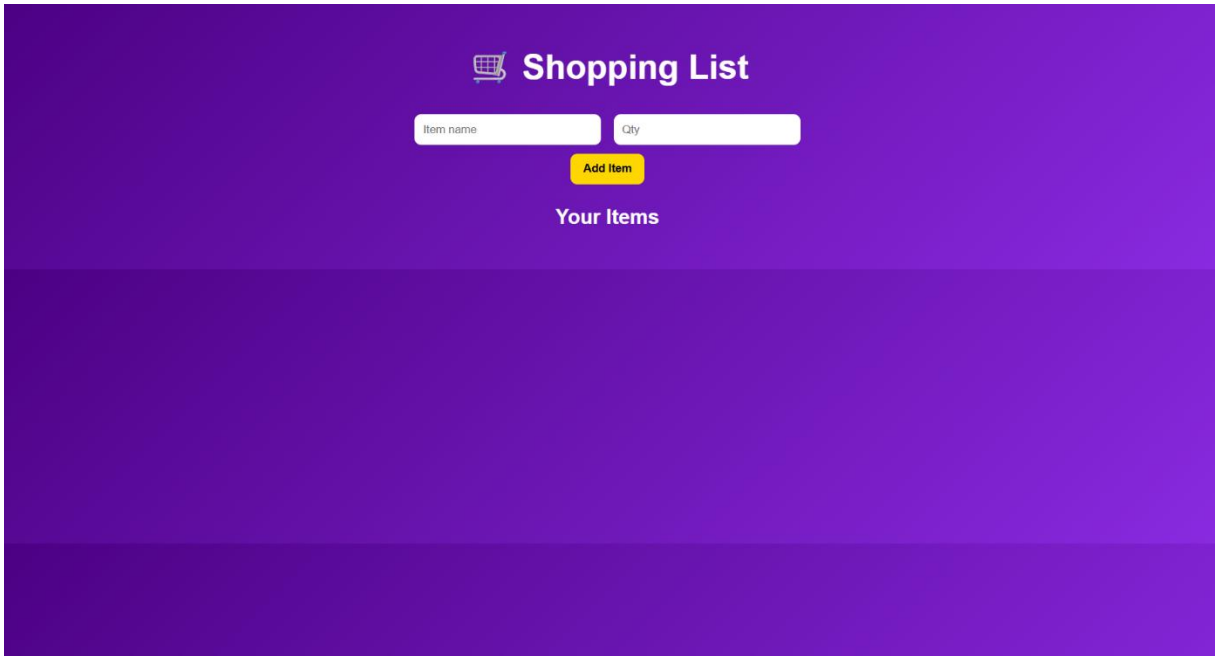
```
document.getElementById("quantity").value;

  await fetch(API + "/add", {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({ name, quantity })
  });

  getItems();
}

async function deleteItem(id) {
  await fetch(API + "/" + id, { method:
"DELETE" });
  getItems();
}

async function editItem(id, oldName, oldQty)
{
  const newName = prompt("Edit name:",
oldName);
  const newQty = prompt("Edit quantity:",
oldQty);

  await fetch(API + "/" + id, {
    method: "PUT",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({ name: newName,
quantity: newQty })
  });

  getItems();
}

getItems();
</script>
</body>
</html>
```

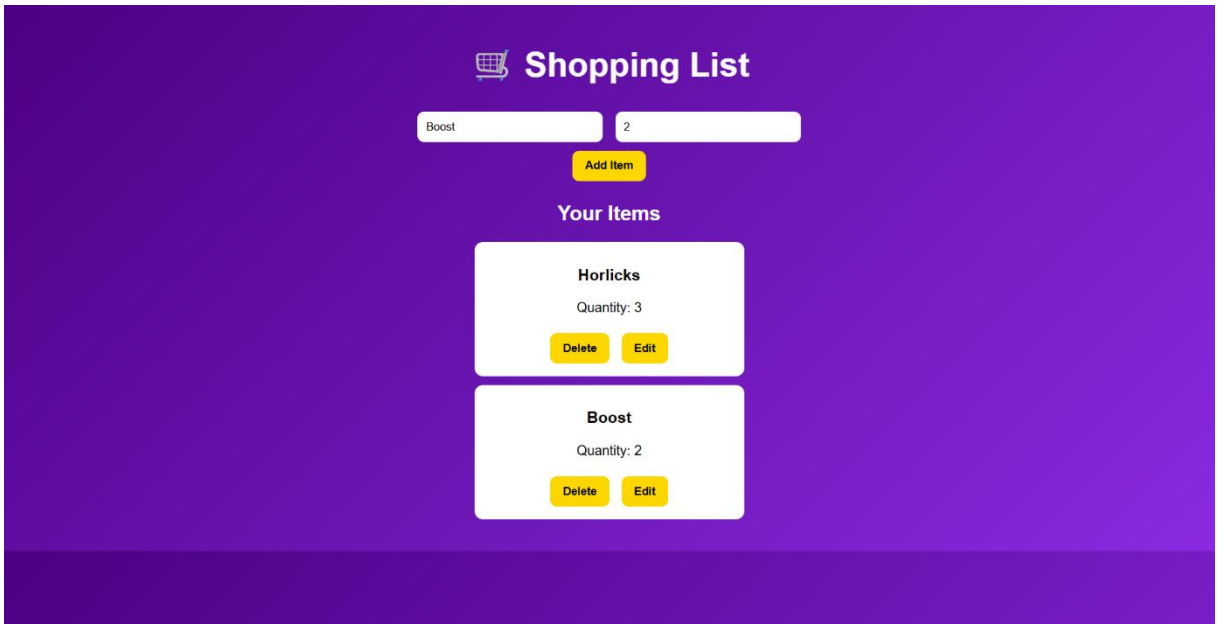
Product Route .js

```
const express = require("express");
const router = express.Router();
const Item = require("../models/Item");

// ADD ITEM
router.post("/add", async (req, res) => {
  try {
    const item = new Item(req.body);
    await item.save();
    res.json(item);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET ALL ITEMS
router.get("/", async (req, res) => {
  try {
    const items = await Item.find();
    res.json(items);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// UPDATE ITEM
router.put("/:id", async (req, res) => {
```

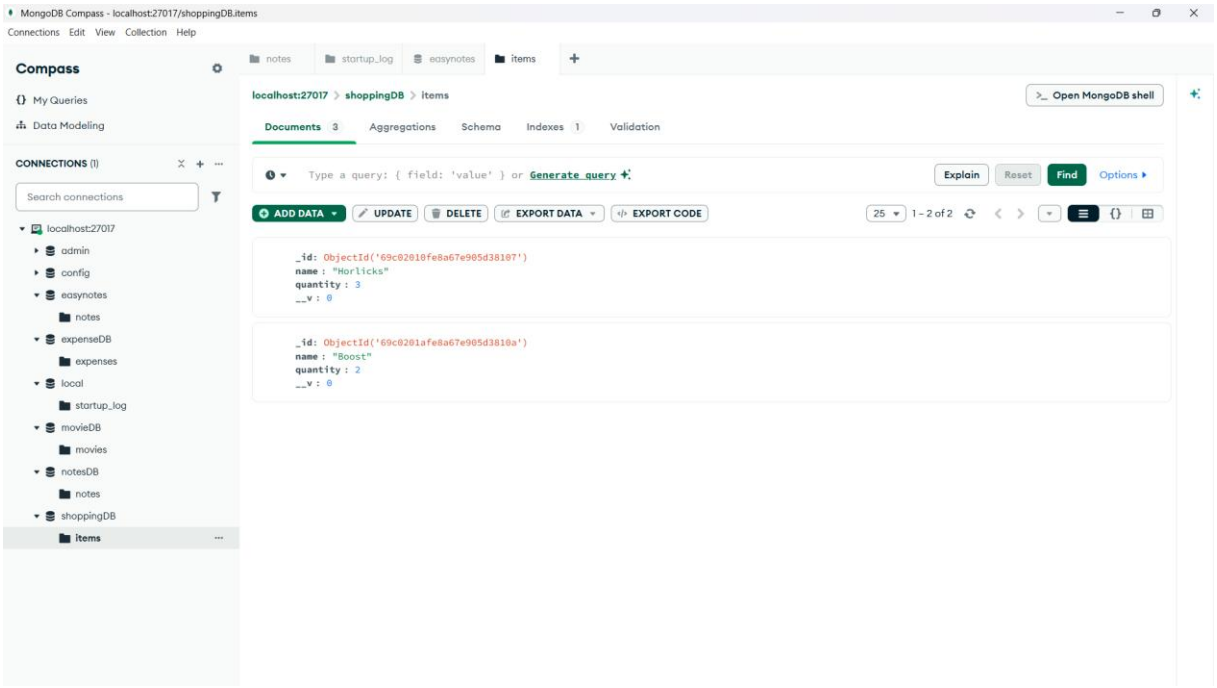
```
  try {
    const updated = await
Item.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updated);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE ITEM
router.delete("/:id", async (req, res) => {
  try {
    await
Item.findByIdAndDelete(req.params.id);
    res.json({ message: "Item deleted" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

OUTPUT :


Shopping List
Item name
Qty
Add Item
Your Items


Shopping List
Boost
2
Add Item
Your Items
Horlicks
Quantity: 3
Delete
Edit
Boost
Quantity: 2
Delete
Edit

Monogo DB


MongoDB Compass - localhost:27017/shoppingDB.items
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
admin
config
easynotes
notes
expenseDB
expenses
local
startup_log
movieDB
movies
notesDB
notes
shoppingDB
items
notes
startup_log
easynotes
items
localhost:27017 > shoppingDB > items
Open MongoDB shell
Documents 3
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
25
1 - 2 of 2
_id: ObjectId('69c02010fe8a67e905d38107')
name : "Horlicks"
quantity : 3
__v : 0
_id: ObjectId('69c0201afe8a67e905d3810a')
name : "Boost"
quantity : 2
__v : 0